

CRICBUZZ LIVESTATS

Real-Time Cricket Insights & SQL-Based Analytics

Executive Summary

Cricbuzz LiveStats is a multi-page cricket analytics application developed to combine live cricket information, structured historical data, SQL-based analysis, and database management in one interactive interface. The application uses Python and Streamlit for the user interface, SQLite as the relational database, SQL for analytical queries, Pandas for tabular processing, Requests for REST API communication, and environment-based configuration for API credentials.

The dashboard is organized into five user-facing pages: Home, Live Matches, Top Player Statistics, SQL Analytics, and CRUD Operations. The SQL module contains 25 analytical questions grouped into Beginner (Q1-Q8), Intermediate (Q9-Q16), and Advanced (Q17-Q25) levels. The CRUD module supports player record creation, reading, updating, and deletion with input and database-integrity validation. The project also handles empty API responses and analytical queries that return no matching rows without crashing the dashboard.

Project at a Glance

Area	Implementation
Frontend / Dashboard	Streamlit multi-page application
Programming	Python
Database	SQLite
Analytics	25 SQL questions + Pandas tables/charts
External Data	REST API integration for live cricket information
Database Management	CRUD operations on player records
Validation	Mandatory fields, duplicate IDs, integrity errors, empty-result handling
Presentation	Metrics, tables, charts, filters, expanders and status messages

Contents

1. Introduction
2. Problem Statement
3. Project Objectives
4. Scope of the Project
5. Technology Stack
6. System Architecture and Workflow
7. Project Folder Structure
8. Database Design
9. Data Collection and API Integration
10. Data Validation and Data Quality
11. Application Modules
12. SQL Analytics Module
13. CRUD Operations
14. Testing and Verification
15. Results and Key Outcomes
16. Challenges and Solutions
17. Limitations
18. Future Enhancements
19. Installation and Execution Guide
20. Deliverables
21. Conclusion

1. Introduction

Cricket generates a large amount of live and historical data, including match status, scores, venues, player performances, team results, batting statistics, and bowling statistics. A useful analytics application must be able to present this information clearly while also supporting structured analysis.

Cricbuzz LiveStats was developed as an interactive cricket analytics and database-learning platform. It brings together REST API integration, relational database concepts, SQL analytics, data visualization, and CRUD operations. The application is designed for learners and users who want to explore cricket information while also understanding how Python, SQL, APIs, and databases work together in a practical project.

2. Problem Statement

Cricket information may be distributed across live API responses and historical database records. Viewing the information alone does not provide an easy way to perform structured SQL analysis, compare player performance, or manipulate database records. Therefore, a unified application is required that can combine live cricket information with a structured relational database and provide interactive analytics through a simple interface.

3. Project Objectives

- Integrate cricket information from a REST API into a Streamlit application.
- Provide a user-friendly page for viewing available live match information and match details.
- Analyze player batting and bowling performance using structured statistics.
- Design and use a relational SQLite database for players, teams, venues, series, matches, and performance statistics.
- Implement 25 SQL analytical questions at Beginner, Intermediate, and Advanced levels.
- Display SQL outputs interactively using tables, metrics, and visualizations.
- Implement Create, Read, Update, and Delete operations for player database management.
- Apply input validation, NULL/data-quality checks, duplicate handling, and database-integrity protection.
- Provide clear handling for empty API responses and SQL queries that return no matching data.

4. Scope of the Project

The implemented scope covers a Streamlit dashboard, live cricket API integration, player-statistics analysis, SQLite database management, 25 SQL analytics questions, and player CRUD operations. The current sample database is intended for demonstration and learning; some advanced historical analyses can legitimately return no rows when the sample records do not satisfy the query conditions.

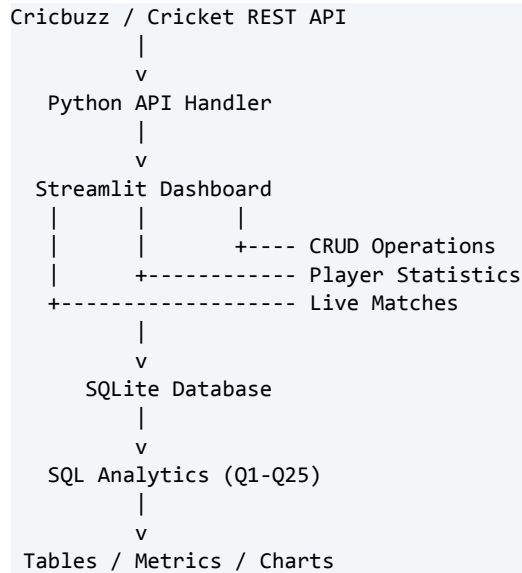
The current CRUD implementation focuses on the players table. Match and statistics tables are available for analytics, but full form-based CRUD for every match/statistics table is not claimed as completed in this report.

5. Technology Stack

Technology	Purpose
Python	Core application logic and integration
Streamlit	Interactive multi-page web dashboard
SQLite	Lightweight relational database
SQL	Filtering, joins, aggregation, CTEs, window functions, ranking and trend analysis
Pandas	DataFrame processing and display
Requests	REST API communication
python-dotenv	Loads API configuration from .env
REST API	Provides available live cricket information

6. System Architecture and Workflow

The high-level data flow of the application is:



API data is retrieved and processed in Python before being displayed in Streamlit. Structured historical and sample records are stored in SQLite. The SQL Analytics page executes queries against SQLite and presents the results as interactive tables and visual summaries.

7. Project Folder Structure

```
Cricbuzz_LiveStats/  
|-- app.py  
|-- requirements.txt  
|-- .env  
|-- .streamlit/  
|   |-- config.toml  
|-- database/  
|   |-- cricket.db  
|   |-- create_tables.py  
|   |-- insert_sample_data.py  
|-- utils/  
|   |-- db_connection.py  
|   |-- api_handler.py  
|   |-- theme.py  
|-- pages/  
|   |-- 1_Live_Matches.py  
|   |-- 2_Top_Player_Stats.py  
|   |-- 3_SQL_Analytics.py  
|   |-- 4_CRUD_Operations.py  
|-- sql/  
|   |-- analytics_queries.py
```

The .env file contains private API configuration and should not be committed to a public repository. The .streamlit/config.toml file controls the application theme without requiring custom HTML/CSS.

8. Database Design

The database is organized into related entities so that match, player, team, venue, series, batting, bowling, and career information can be analyzed using SQL joins and aggregations.

Table	Purpose	Important Fields
players	Stores player profile information	player_id, full_name, country, playing_role, batting_style, bowling_style
teams	Stores cricket teams	team_id, team_name, country
venues	Stores match venues	venue_id, venue_name, city, country, capacity
series	Stores series information	series_id, series_name, host_country, match_type, dates, total_matches
matches	Stores match-level information	teams, venue, series, date, format, status, winner, margin, toss
batting_stats	Stores innings-level batting performance	runs, balls, fours, sixes, strike_rate, not_out
bowling_stats	Stores innings-level bowling performance	overs, runs_conceded, wickets, economy_rate
player_career_stats	Stores format-wise career summaries	runs, averages, strike_rate, wickets, catches, stumpings

8.1 Main Relationships

- `batting_stats.player_id` and `bowling_stats.player_id` relate performance records to players.
- `matches` references `team1`, `team2`, `venue`, `series`, `winning team`, and `toss winner`.
- `player_career_stats` relates format-wise career summaries to players.
- The relational structure supports joins across players, matches, teams, venues, and statistics.

9. Data Collection and API Integration

The Live Matches module communicates with a cricket REST API using Python Requests. API credentials are loaded from an environment file rather than being hard-coded into the application. The returned JSON data is parsed and displayed through Streamlit components.

Because live sports data changes continuously, the exact information visible on the dashboard depends on the matches and fields returned by the selected API endpoint at runtime. The application should therefore handle missing fields, empty responses, and API errors gracefully.

9.1 API Configuration

```
# .env
RAPIDAPI_KEY=your_api_key_here
```

The actual API key must remain private and should be excluded from GitHub using `.gitignore`.

10. Data Validation and Data Quality

Data validation is applied at the user-input, database, and API-processing levels. The objective is to reduce invalid records and prevent poor-quality data from affecting SQL analytics.

Validation Type	Example in the Project	Purpose
Mandatory-field validation	Player name and country cannot be blank	Prevents incomplete player records
Uniqueness validation	Duplicate <code>player_id</code> raises an integrity error	Protects primary-key uniqueness
NULL handling	<code>dropna()</code> , <code>fillna()</code> , SQL NULL checks	Prevents missing values from breaking summaries
Range/data-quality checks	Runs, balls, wickets and similar measures should not be negative	Prevents logically invalid statistics
Referential integrity	Related records may prevent unsafe deletion	Preserves database relationships
API validation	Check response/fields before display	Prevents missing API data from crashing the page
Empty-result handling	SQL page can display 'No Data'	Separates valid zero-row results from query errors

10.1 Example Input Validation

```
if not full_name.strip():
    st.warning("Player name is required.")
elif not country.strip():
    st.warning("Country is required.")
```

10.2 Example Data-Quality SQL Checks

```
SELECT *
FROM players
WHERE full_name IS NULL
    OR country IS NULL;

SELECT *
FROM batting_stats
WHERE runs < 0
    OR balls < 0;

SELECT player_id, COUNT(*)
FROM players
GROUP BY player_id
HAVING COUNT(*) > 1;
```

11. Application Modules

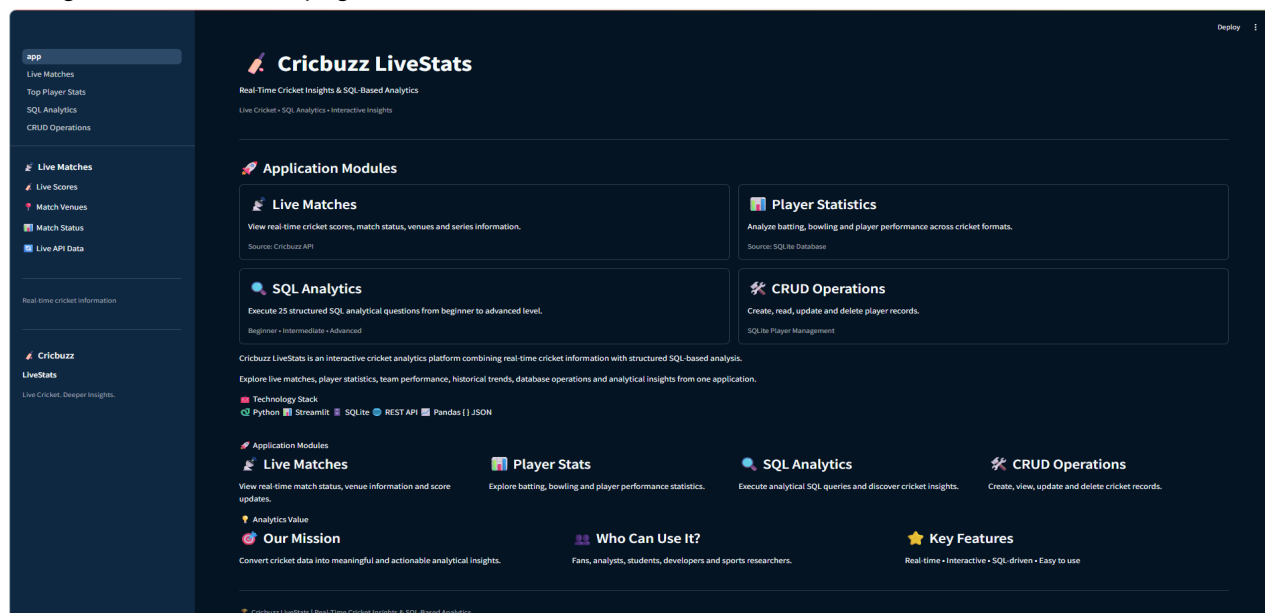
Streamlit App Link :

[https://cricbuzz-livestats-9ey9sxxw894kvronf5grjg9.streamlit.app/CRUD_Operations]

11.1 Home Page

The Home page introduces Cricbuzz LiveStats and summarizes the application. It presents the technology stack, major modules, project purpose, and key dashboard indicators. Streamlit's built-in navigation is used to move between pages.

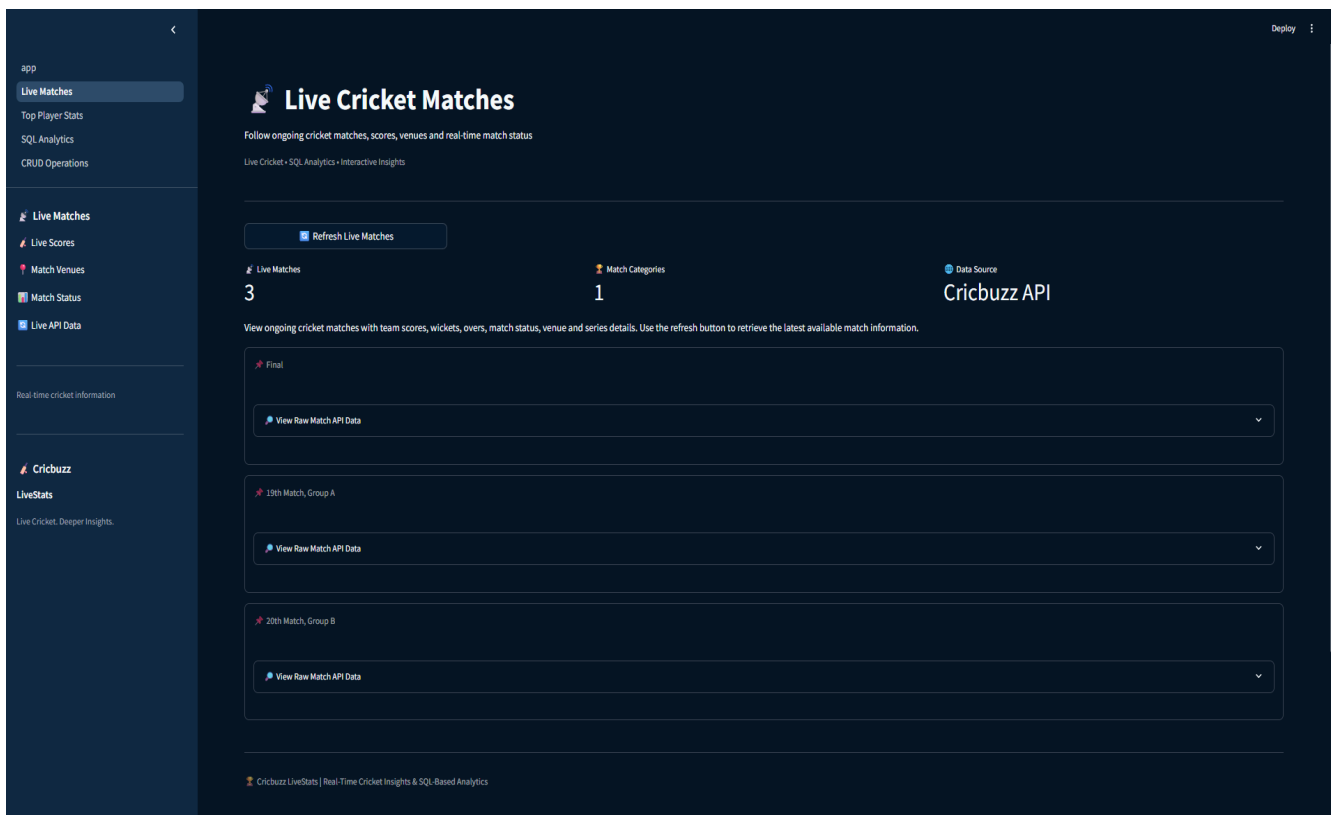
- Project title and overview
- Technology stack
- Application modules
- Dashboard KPIs
- Navigation to functional pages



11.2 Live Matches Page

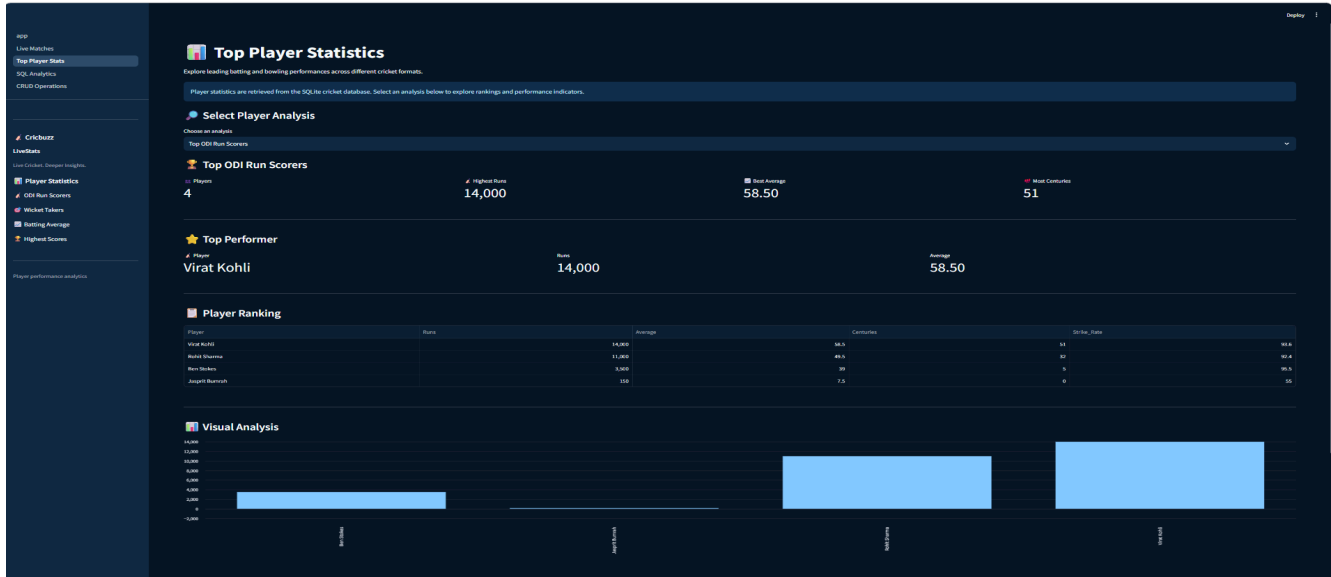
The Live Matches page is the API-driven part of the project. It is designed to display the live or currently available match information returned by the cricket API, such as participating teams, match status, scores, series information, and venue details when those fields are available.

When the API does not return a live fixture, the page should show an informative message rather than a program error.



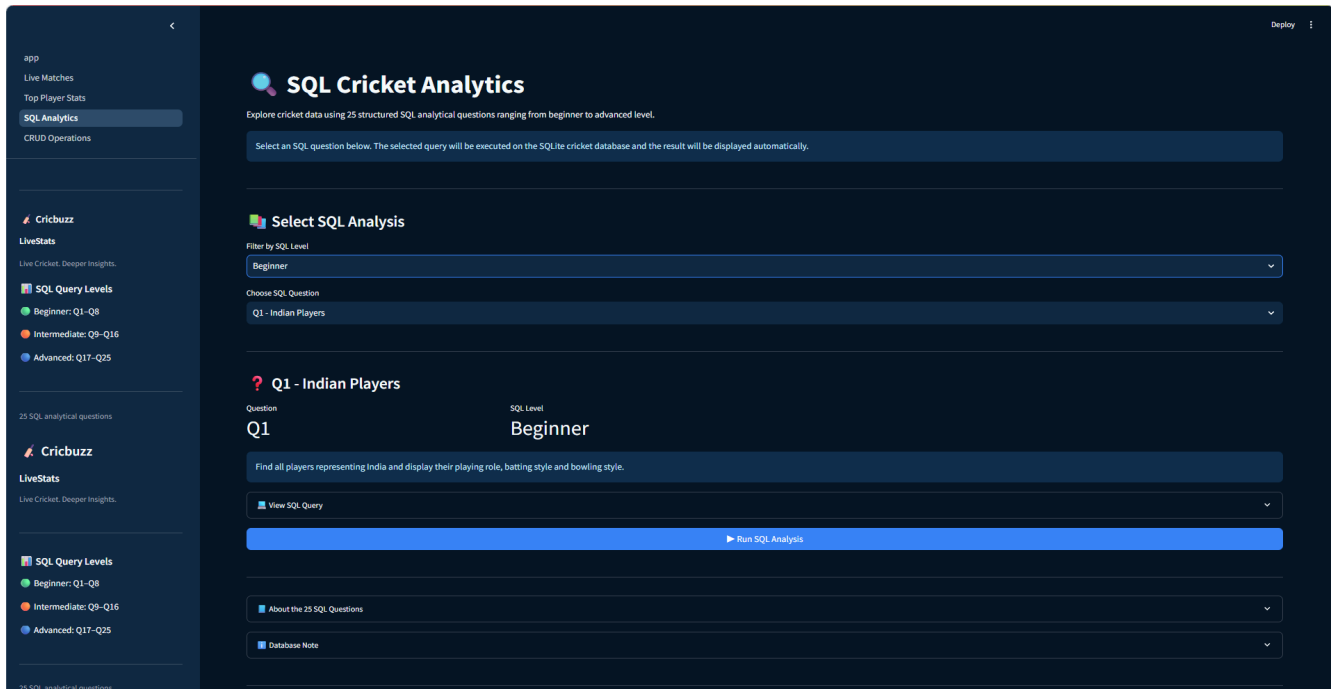
11.3 Top Player Statistics Page

The Top Player Statistics page presents player-performance analysis in an easy-to-read format. It supports categories such as run scoring, highest scores, batting averages, wicket taking, and other available performance measures. Results are displayed through ranking tables, metrics, and charts.



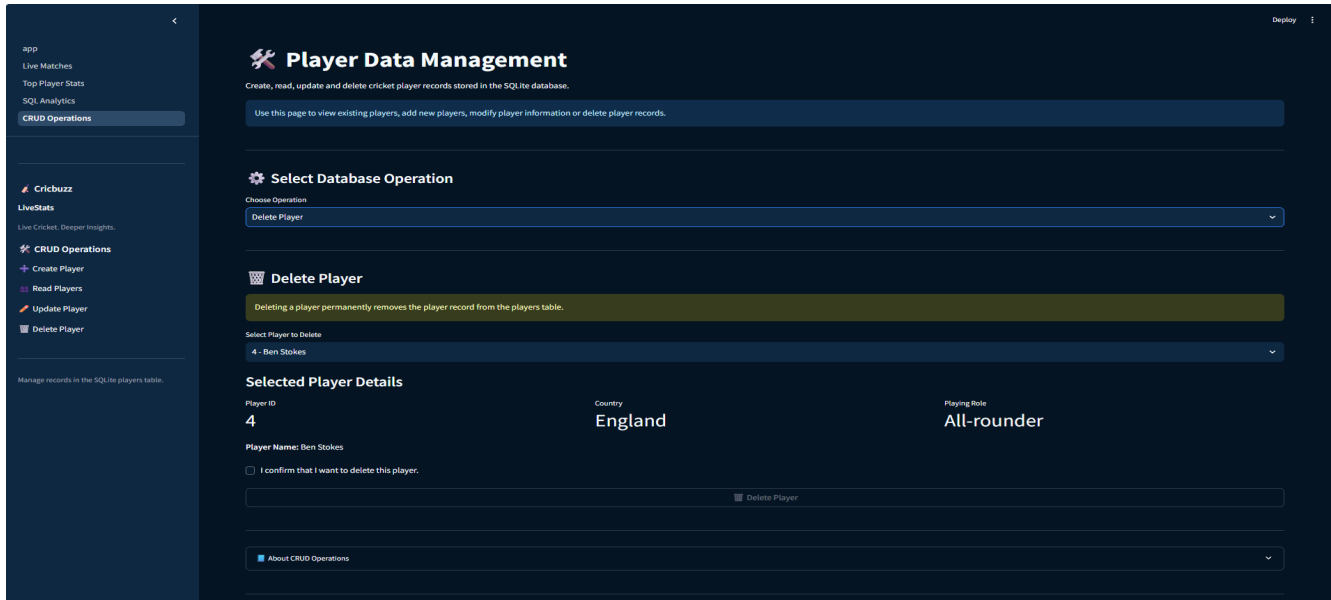
11.4 SQL Analytics Page

The SQL Analytics page is the main database-analysis component. Users select a difficulty level and analytical question, inspect the SQL statement, execute it, and view the resulting table. The page distinguishes successful queries with zero matching rows from actual SQL errors.



11.5 CRUD Operations Page

The CRUD page manages records in the players table. It supports Create, Read, Update, and Delete operations through Streamlit forms and controls. The Read view also summarizes the database using KPI metrics, player tables, role distribution, country distribution, and CSV download.



12. SQL Analytics Module

The project contains 25 analytical questions divided into three levels. This organization demonstrates a progression from basic SQL operations to more advanced analytical techniques.

Level	Questions	Main SQL Concepts
Beginner	Q1-Q8	SELECT, WHERE, ORDER BY, GROUP BY, JOIN, aggregate functions, date filtering
Intermediate	Q9-Q16	Multi-table joins, conditional logic, home/away analysis, date aggregation, self-joins
Advanced	Q17-Q25	CTEs, window functions, ranking, trend analysis, weighted scores, recent form, variance

12.1 Query Coverage

Query	Analytical Purpose
Q1	Indian players and profile details
Q2	Recent matches with teams and venues
Q3	Top ODI run scorers
Q4	Large-capacity venues
Q5	Team win counts
Q6	Player count by role
Q7	Highest score by format
Q8	Series started in 2024
Q9	All-rounders with >1000 runs and >50 wickets
Q10	Last 20 completed matches
Q11	Multi-format player run totals and batting averages
Q12	Team home-versus-away win performance
Q13	Consecutive batting-position pair analysis
Q14	Bowling performance at the same venue
Q15	Player performance in close matches
Q16	Year-wise batting performance since 2020
Q17	Toss decision and match-winning advantage
Q18	Economical ODI/T20 bowling analysis
Q19	Batting consistency using average runs and variance

Q20	Multi-format match counts and batting averages
Q21	Weighted batting, bowling and fielding ranking
Q22	Head-to-head team analysis
Q23	Recent player form and consistency variance
Q24	Repeated consecutive-batsmen pair analysis
Q25	Quarterly performance trend and career trajectory

12.2 SQLite Compatibility Adjustment

During testing, Q19 and Q23 initially used SQRT() to calculate standard deviation. The local SQLite environment reported 'no such function: SQRT'. To keep the queries portable, the consistency measure was changed to variance:

```
Variance = AVG(runs * runs) - AVG(runs) * AVG(runs)
```

Lower variance indicates more consistent run scoring, while higher variance indicates greater fluctuation. After this adjustment, Q19 and Q23 executed successfully. They may still return zero rows when the sample database does not contain enough qualifying historical performances.

13. CRUD Operations

Operation	Implementation	Validation
Create	Form inserts a new player into players	Mandatory name/country; unique player_id
Read	Displays players, KPIs, role/country analysis and CSV download	Handles empty table safely
Update	Selects a player and updates editable fields	Pre-populates current values; validates required fields
Delete	Deletes selected player after confirmation	Handles database integrity errors

13.1 CRUD Workflow

```
Create Test Player -> Read Record -> Update Record -> Delete Record -> Verify Removal
```

Parameterized SQL statements are used for insert, update, and delete operations. This avoids constructing SQL commands by concatenating raw user input.

14. Testing and Verification

Final functional testing should verify each Streamlit page and the main database/API paths. A query is considered successfully executed when it appears in the selector, its SQL is displayed, execution produces no SQL/Python error, and the result is shown either as records or as a valid 'No Data' state.

Test Area	Verification
Home	Page loads; no duplicate headings/footer; navigation works
Live Matches	API call handled; match fields displayed when available; empty/error response handled
Top Player Stats	Selectors, ranking tables, metrics and charts work
SQL Q1-Q25	Each query executes without unexpected SQL/Python errors
Q19/Q23	Variance-based versions execute without SQRT error
CRUD Create	New temporary player can be added
CRUD Read	Added record appears; KPIs/table refresh
CRUD Update	Temporary player can be modified
CRUD Delete	Temporary player can be removed; integrity errors handled

Validation	Blank required fields and duplicate IDs produce user-friendly messages
------------	--

15. Results and Key Outcomes

- A multi-page Streamlit cricket analytics dashboard was developed.
- REST API integration was incorporated for live/available cricket information.
- A relational SQLite schema was created for players, teams, venues, series, matches, batting, bowling, and career statistics.
- Twenty-five SQL analytical questions were organized into Beginner, Intermediate, and Advanced levels.
- Player database CRUD operations were implemented through a form-based interface.
- Player records can be summarized using KPIs, tables, role distributions, and country distributions.
- Input, database-integrity, empty-result, and API-response handling improve robustness.
- SQLite compatibility issues in advanced consistency queries were identified and corrected.

15.1 Interpretation of 'No Data'

A 'No Data' result does not necessarily mean that an SQL query is incorrect. Advanced questions often require a minimum number of matches, innings, overs, quarters, or historical records. When the sample database does not meet those conditions, a correct query can legitimately return zero rows. The dashboard therefore reports the empty result separately from an execution error.

16. Challenges and Solutions

Challenge	Solution
Different/missing API fields	Use safe response handling and display only available fields
No live fixture at runtime	Show an informative empty-state message
SQLite SQRT unavailable in Q19/Q23	Use variance as the consistency measure
Advanced queries need more history	Treat zero-row output as valid when conditions are not met
Duplicate player IDs	Catch <code>sqlite3.IntegrityError</code> and ask for a unique ID
Deleting referenced records	Catch integrity errors and preserve database relationships
Duplicate sidebar/footer UI during development	Centralize theme/sidebar/footer helpers and call each once
Raw HTML appearing in Streamlit	Use native Streamlit components and <code>config.toml</code> theme settings

17. Limitations

- The sample SQLite database is small compared with a production cricket data warehouse.
- Some advanced questions require more historical records than the sample database currently contains.
- The exact Live Matches fields depend on the selected REST API endpoint and its runtime response.
- The current CRUD interface focuses on player records rather than full CRUD for every match/statistics table.
- Some analyses such as true partnership runs would require richer ball-by-ball or dismissal data for exact calculation.

18. Future Enhancements

- Increase the historical dataset and automate periodic API-to-database synchronization.
- Add richer scorecards and more detailed player/team comparisons when supported by the API.
- Extend CRUD operations to matches and selected statistics tables.
- Add ball-by-ball data for exact partnership and innings-level analyses.
- Deploy the Streamlit application to a cloud platform.
- Add predictive analytics or machine-learning models for player form and match outcomes.
- Add user-selected date ranges, teams, formats, and venues for deeper interactive filtering.

19. Installation and Execution Guide

19.1 Prerequisites

- Python 3.x
- Internet connection for API-based pages
- Valid API subscription/key
- VS Code or another Python IDE (optional)

19.2 Install Dependencies

```
python -m pip install streamlit pandas requests python-dotenv
```

19.3 Configure API Key

```
RAPIDAPI_KEY=your_api_key_here
```

19.4 Initialize Database

If the database must be recreated, run the schema and sample-data scripts according to the project setup:

```
python database/create_tables.py
python database/insert_sample_data.py
```

19.5 Run the Application

```
python -m streamlit run app.py
```

The Streamlit application opens in a browser. Use the sidebar to navigate between Live Matches, Top Player Stats, SQL Analytics, and CRUD Operations.

20. Project Deliverables

Deliverable	Status / Description
Source Code	Python Streamlit application with modular pages and utilities
Database Schema	SQLite tables for cricket entities and statistics
Sample Data	Demonstration records for SQL analytics

SQL Practice	25 analytical questions across three levels
CRUD Module	Player Create, Read, Update and Delete
Requirements	Python dependency list
API Configuration	Environment-based API key setup
Documentation	This full project report and setup guidance
Demo	Working Streamlit dashboard for final recording

21. Conclusion

Cricbuzz LiveStats demonstrates how live API integration, relational database design, SQL analytics, data visualization, and CRUD operations can be combined in a single Python project. The application provides a practical environment for exploring cricket information while also demonstrating database concepts from basic filtering and aggregation to CTEs, window functions, ranking, and trend analysis.

The project successfully establishes the core dashboard, SQLite schema, player statistics, 25-query analytics module, player CRUD workflow, and data-validation mechanisms. The remaining opportunities are mainly extensions rather than core requirements: a larger historical dataset, richer API coverage, additional CRUD entities, cloud deployment, and predictive analytics.

Appendix A - Recommended Final Submission Checklist

- Run all pages once and confirm there are no unexpected Python errors.
- Run Q1-Q25 and confirm each produces either records or a valid No Data state.
- Test CRUD with a temporary player: Create -> Read -> Update -> Delete.
- Verify the API key is not hard-coded anywhere.
- Ensure .env is listed in .gitignore before GitHub upload.
- Capture clean screenshots of Home, Live Matches, Player Stats, SQL Analytics, and CRUD.
- Finalize README.md and requirements.txt.
- Record the final project demonstration only after the code is frozen.